

Deep Learning

Referencia: Bishop, D2L e MIT conforme a lista de aulas.

01. Introduction

Deep learning aprende representacoes em camadas:

$$x \xrightarrow{f_1} h_1 \xrightarrow{f_2} h_2 \rightarrow \dots \rightarrow \hat{y}$$

A ideia central e ajustar parametros θ para minimizar uma perda:

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_{\theta(x_i)}, y_i)$$

Notas rapidas:

- Dados viram tensores.
- O modelo produz predicoes.
- A loss mede o erro.
- O otimizador altera os pesos para reduzir a loss.
- Generalizar significa ir bem em dados nao vistos, nao apenas memorizar treino.

```
import torch
```

```
x = torch.randn(32, 10)      # batch com 32 exemplos e 10 atributos  
y = torch.randint(0, 3, (32,))
```

```
print(x.shape, y.shape)
```

02. Basics on Neural Networks

Um neuronio combina entrada, pesos e vies:

$$z = w^T x + b$$

Depois aplica uma ativacao nao linear:

$$h = \varphi(z)$$

Sem nao linearidade, varias camadas viram apenas uma transformacao linear.

MLP:

$$h_1 = \varphi(W_1 x + b_1), \quad \hat{y} = W_2 h_1 + b_2$$

Para classificacao multiclasse, usamos softmax:

$$p(y = k \mid x) = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

Cross-entropy:

$$\mathcal{L} = -\log p(y = c \mid x)$$

```
import torch  
from torch import nn
```

```

model = nn.Sequential(
    nn.Linear(10, 64),
    nn.ReLU(),
    nn.Linear(64, 3)
)

x = torch.randn(32, 10)
y = torch.randint(0, 3, (32,))

logits = model(x)
loss = nn.CrossEntropyLoss()(logits, y)

loss.backward()
print(loss.item())

```

03. Convolutional Neural Networks + CNN Architectures

CNNs exploram estrutura espacial. Em vez de conectar tudo com tudo, filtros pequenos percorrem a imagem.

Convolucao 2D simplificada:

$$Y_{i,j,k} = b_k + \sum_c \sum_u \sum_v W_{u,v,c,k} X_{i+u,j+v,c}$$

Intuicao:

- Filtros aprendem detectores locais: bordas, texturas, partes.
- O mesmo filtro e reutilizado em varias posicoes: compartilhamento de pesos.
- Isso reduz parametros e cria equivariancia a translacao.

Tamanho da saida em uma dimensao:

$$O = \left\lfloor \frac{I + 2P - K}{S} \right\rfloor + 1$$

onde I e entrada, P padding, K kernel e S stride.

```

import torch
from torch import nn

cnn = nn.Sequential(
    nn.Conv2d(3, 16, kernel_size=3, padding=1),
    nn.ReLU(),
    nn.MaxPool2d(2),
    nn.Conv2d(16, 32, kernel_size=3, padding=1),
    nn.ReLU(),
    nn.AdaptiveAvgPool2d((1, 1)),
    nn.Flatten(),
    nn.Linear(32, 10)
)

x = torch.randn(8, 3, 64, 64)
logits = cnn(x)
print(logits.shape) # torch.Size([8, 10])

```

Arquiteturas importantes:

- LeNet: CNN classica pequena para digitos.

- AlexNet: ReLU, GPU, dropout; marco em ImageNet.
- VGG: muitos filtros 3×3 , arquitetura simples e profunda.
- Inception/GoogLeNet: varios tamanhos de filtro em paralelo.
- ResNet: conexoes residuais facilitam redes muito profundas.

Bloco residual:

$$y = F(x) + x$$

```
import torch
from torch import nn

class ResidualBlock(nn.Module):
    def __init__(self, channels):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(channels, channels, 3, padding=1),
            nn.BatchNorm2d(channels),
            nn.ReLU(),
            nn.Conv2d(channels, channels, 3, padding=1),
            nn.BatchNorm2d(channels),
        )
        self.act = nn.ReLU()

    def forward(self, x):
        return self.act(self.net(x) + x)
```

04. Training

Treinar e repetir:

1. forward: calcular $\hat{y} = f_{\theta(x)}$
2. loss: medir $\mathcal{L}(\hat{y}, y)$
3. backward: calcular gradientes $\nabla_{\theta} \mathcal{L}$
4. update: alterar pesos

Atualizacao basica por gradiente descendente:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t)$$

Mini-batch SGD aproxima o gradiente usando poucos exemplos:

$$g_t = \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_{\theta} \mathcal{L}_{i(\theta_t)}$$

Pontos praticos:

- Normalizar entradas ajuda a estabilizar treino.
- Inicializacao ruim pode matar gradientes.
- BatchNorm reduz sensibilidade a escala interna.
- Learning rate costuma importar mais que pequenos detalhes do modelo.

```
import torch
from torch import nn

model = nn.Sequential(nn.Linear(10, 64), nn.ReLU(), nn.Linear(64, 3))
opt = torch.optim.SGD(model.parameters(), lr=0.1)
loss_fn = nn.CrossEntropyLoss()
```

```

for step in range(100):
    x = torch.randn(32, 10)
    y = torch.randint(0, 3, (32,))

    opt.zero_grad()
    logits = model(x)
    loss = loss_fn(logits, y)
    loss.backward()
    opt.step()

```

05. Optimization & Generalization

Otimizacao pergunta: como reduzir a loss de treino?

Generalizacao pergunta: o modelo aprendeu padroes ou decorou?

Momentum suaviza atualizacoes:

$$v_{t+1} = \beta v_t + \nabla_{\theta} \mathcal{L}(\theta_t)$$

$$\theta_{t+1} = \theta_t - \eta v_{t+1}$$

Adam combina media de gradientes e de gradientes ao quadrado:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Regularizacao L2:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \lambda \|\theta\|_2^2$$

Dropout:

$$\tilde{h} = m \cdot h, \quad m_i \sim \text{Bernoulli}(p)$$

```

import torch
from torch import nn

```

```

model = nn.Sequential(
    nn.Linear(100, 256),
    nn.ReLU(),
    nn.Dropout(p=0.5),
    nn.Linear(256, 10)
)

```

```

opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=50)

```

Checklist mental:

- loss de treino alta: underfitting, modelo fraco, LR ruim ou poucos epochs.
- treino bom e validacao ruim: overfitting.
- treino instavel: LR alto, dados mal escalados, batch pequeno ou inicializacao ruim.
- validacao melhorando: continue.
- validacao parou de melhorar: early stopping ou reduzir LR.

06. Semantic Segmentation

Segmentacao semantica classifica cada pixel.

Entrada:

$$X \in \mathbb{R}^{C \times H \times W}$$

Saida:

$$\hat{Y} \in \mathbb{R}^{K \times H \times W}$$

onde K e o numero de classes. Cada pixel recebe uma distribuicao sobre classes.

Loss por pixel:

$$\mathcal{L} = -\frac{1}{HW} \sum_{i,j} \log p(y_{i,j} | x)$$

IoU para uma classe:

$$\text{IoU} = \frac{\text{TP}}{\text{TP} + \text{FP} + \text{FN}}$$

Dice:

$$\text{Dice} = \frac{2|A \cap B|}{|A| + |B|}$$

Arquiteturas comuns:

- FCN: troca camadas densas por convolucoes e gera mapa denso.
- U-Net: encoder comprime contexto, decoder recupera resolucao, skips preservam detalhe.
- DeepLab: convolucoes dilatadas aumentam campo receptivo sem perder tanta resolucao.

```
import torch
from torch import nn

class TinyUNet(nn.Module):
    def __init__(self, classes):
        super().__init__()
        self.enc = nn.Sequential(
            nn.Conv2d(3, 16, 3, padding=1),
            nn.ReLU(),
            nn.Conv2d(16, 16, 3, padding=1),
            nn.ReLU(),
        )
        self.pool = nn.MaxPool2d(2)
        self.dec = nn.Sequential(
            nn.ConvTranspose2d(16, 16, 2, stride=2),
            nn.ReLU(),
            nn.Conv2d(16, classes, 1)
        )

    def forward(self, x):
        h = self.enc(x)
        z = self.pool(h)
        return self.dec(z)

model = TinyUNet(classes=4)
```

```
x = torch.randn(2, 3, 128, 128)
y = torch.randint(0, 4, (2, 128, 128))

logits = model(x)
loss = nn.CrossEntropyLoss()(logits, y)
print(logits.shape, loss.item())
```

Resumo final

Deep learning é a composição de funções parametrizadas. Redes densas aprendem relações globais; CNNs exploram localidade e compartilhamento de pesos; treinamento ajusta parâmetros via gradientes; otimização busca reduzir a loss; generalização exige controlar overfitting; segmentação leva classificação do nível da imagem para o nível do pixel.